

Python toolbox

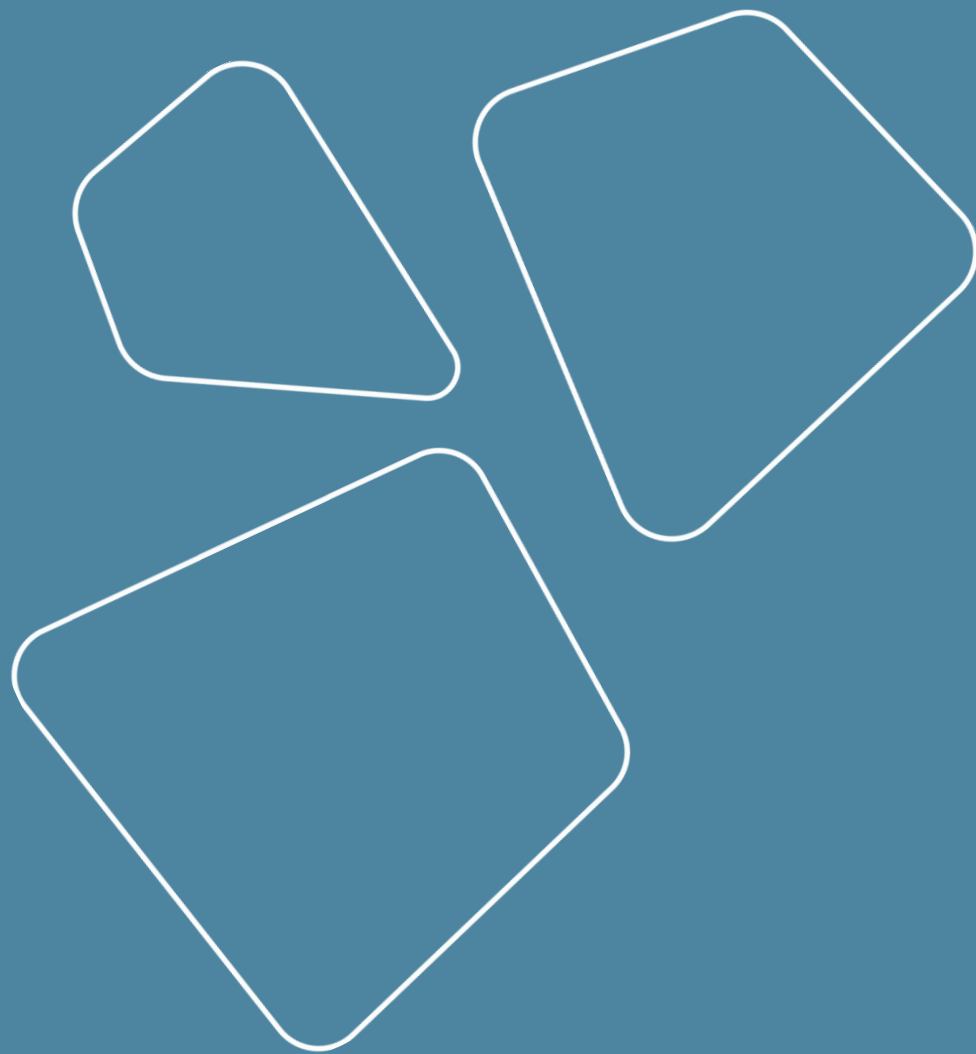
Vladislav Goncharenko



MIPT
spring 2026

Outline

1. Python project structure
2. Managing environments
3. Managing dependencies
4. CLI
5. Changelog



REPL

Read-eval-print loop is an interactive toplevel or language shell, is a simple interactive computer programming environment that takes single user inputs, executes them, and returns the result to the user; a program written in a REPL environment is executed piecewise.

Python project structure

girafe
ai

01

Modules in Python

The simplest module is a file containing Python definitions and statements.

Definitions from a module can be imported into other modules or into the main module.

```
# Fibonacci numbers module

def fib(n):    # write Fibonacci series up to n
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):   # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

```
>>> import fibo

>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

pyproject.toml

Standard file format for modern package metadata instead of many older files.

[Packaging user guide](#), [Pip documentation on pyproject](#)

[Set of tools supporting pyproject.toml](#)

Examples:

- [one](#)
-

```
[project]
name = "mypackage"
version = "2023.05.1"
description = "A package for demonstrating Python packaging"

[build-system]
requires = ["setuptools >= 61.0.0"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
```

Packages

- [python package structure](#)
 - common extensible format
 - able to be packaged
- subject to build systems

[More on python packaging](#)

[Official packaging tutorial](#)

sound/	Top-level package
__init__.py	Initialize the sound package
formats/	Subpackage for file format conversions
__init__.py	
wavread.py	
wavwrite.py	
aiffread.py	
aiffwrite.py	
auread.py	
auwrite.py	
...	
effects/	Subpackage for sound effects
__init__.py	
echo.py	
surround.py	
reverse.py	
...	
filters/	Subpackage for filters
__init__.py	
equalizer.py	
vocoder.py	
karaoke.py	
...	

How to organize subpackages

Usually people group entities by their functionality and API e.g.:

- datasets (and tools such as DataLoaders)
- models
- metrics
- visualizations

Not by iteration of your experiment e.g. baseline, main solution.

Packaging options in uv

<https://docs.astral.sh/uv/concepts/projects/init/>

Aspect	Application	Packaged application	Library
Intended use	Web servers, scripts, CLI run from project source	Publishable CLIs or projects with dedicated tests	Importable code consumed by other projects
Main interface	Script file, e.g. <code>main.py</code>	Installed command entry point	Python import API
Default source layout	Flat	Src	Src
Needs build system	No	Yes	Yes
Has <code>[project.scripts]</code>	No	Yes	No
Has <code>py.typed</code> file	No	No	Yes



py.typed file

`py.typed` is a marker file defined by PEP 561. It tells type checkers such as mypy, Pyright, Pylance, etc. that your distributed package contains usable type information.

Without `py.typed`, a downstream user's type checker may treat your installed package as effectively untyped, even though annotations exist in the source. The official typing guide says tools will not treat a package as providing type information unless it includes `py.typed`.

<https://peps.python.org/pep-0561/>

What the file contains: nothing. You just create an empty file.

Multi-package monorepo

Sometimes it is handful to have single repository for several separate packages. Examples are:

- company repository
 - departments having separate subpackages but sharing common tools
- product repository
 - consumer site backend separated from seller backend while manipulating on intersecting set of DBs
 - chat bot backend separated from other backends

flat layout vs src layout

For more complex projects (e.g. with multiple [import packages](#))

```
.
├── README.md
├── noxfile.py
├── pyproject.toml
├── setup.py
├── awesome_package/
│   ├── __init__.py
│   └── module.py
└── tools/
    ├── generate_awesome.py
    └── decrease_world_suck.py
```

```
.
├── README.md
├── noxfile.py
├── pyproject.toml
├── setup.py
├── src/
│   └── awesome_package/
│       ├── __init__.py
│       └── module.py
└── tools/
    ├── generate_awesome.py
    └── decrease_world_suck.py
```

```
albatross
├── packages
│   ├── bird-feeder
│   │   ├── pyproject.toml
│   │   └── src
│   │       └── bird_feeder
│   │           ├── __init__.py
│   │           └── foo.py
│   └── seeds
│       ├── pyproject.toml
│       └── src
│           └── seeds
│               ├── __init__.py
│               └── bar.py
├── pyproject.toml
├── README.md
├── uv.lock
└── src
    └── albatross
        └── main.py
```

flat layout vs src layout

In “flat layout” various configuration files and import packages are all in the top-level directory.

In “src layout” code that is intended to be importable is moved into a `src/` subdirectory.

- The src layout requires installation of the project to be able to run its code, and the flat layout does not.
- The src layout helps prevent accidental usage of the in-development copy of the code.
- The src layout helps enforce that an editable installation is only able to import files that were meant to be importable.

[Packaging guide](#), [setuptools autodiscovery](#), [workspaces in uv](#)

Import system

Is actually not straightforward =(

Where to import

Basically **at the start of a file**.

Proper import should not have side effects, so it is fast and furious.

The most canonical way is to import a module rather than particular object (e.g. class, function, etc.). This helps to avoid circular imports because module in outer context should not have all the components at import time.

Exceptions

Don't catch pure `Exception`!

`Exception` can be used as a wildcard that catches (almost) everything. However, it is good practice to be as specific as possible with the types of exceptions that we intend to handle, and to allow any unexpected exceptions to propagate on.

More info [here](#) and [there](#)

```
import sys

try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("OS error:", err)
except ValueError:
    print("Could not convert data to an integer.")
except Exception as err:
    print(f"Unexpected {err=}, {type(err)=}")
    raise
```

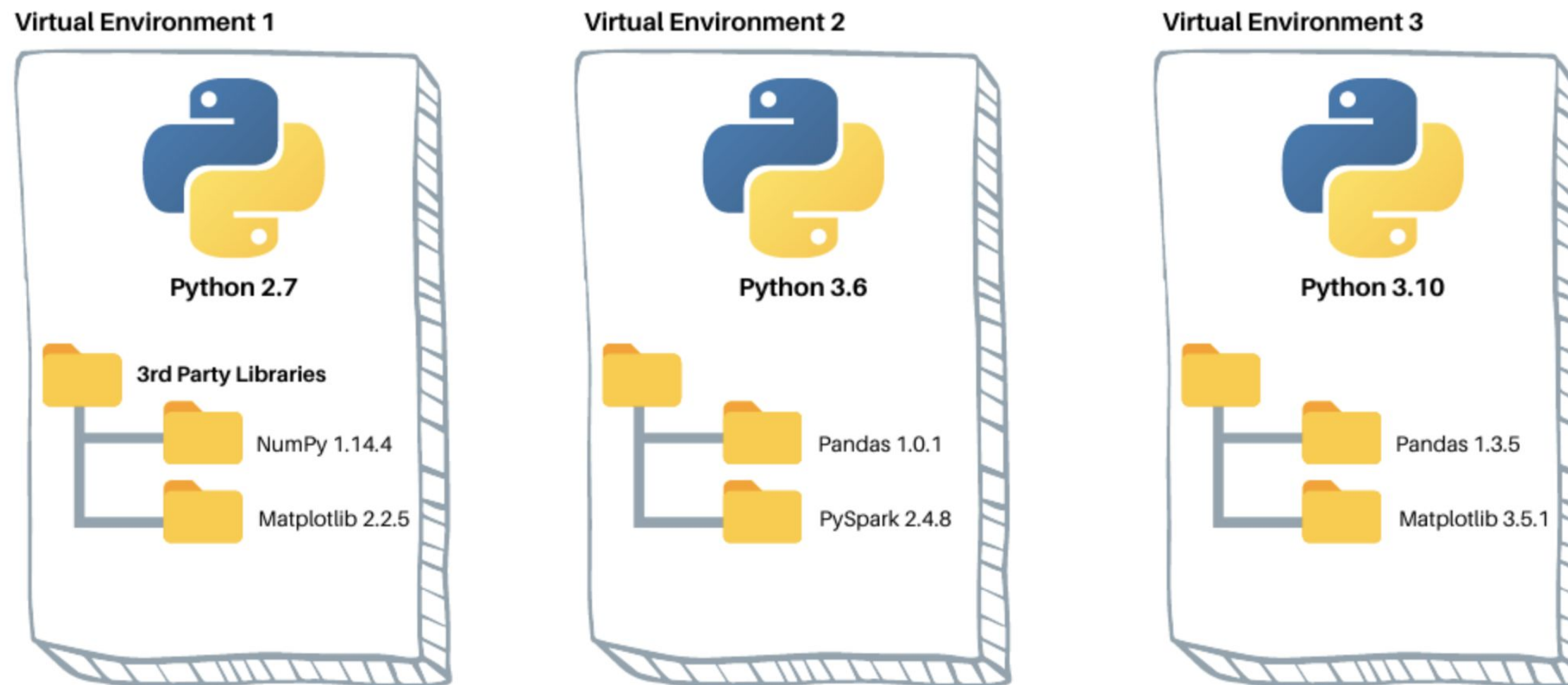
Environments

girafe
ai

02

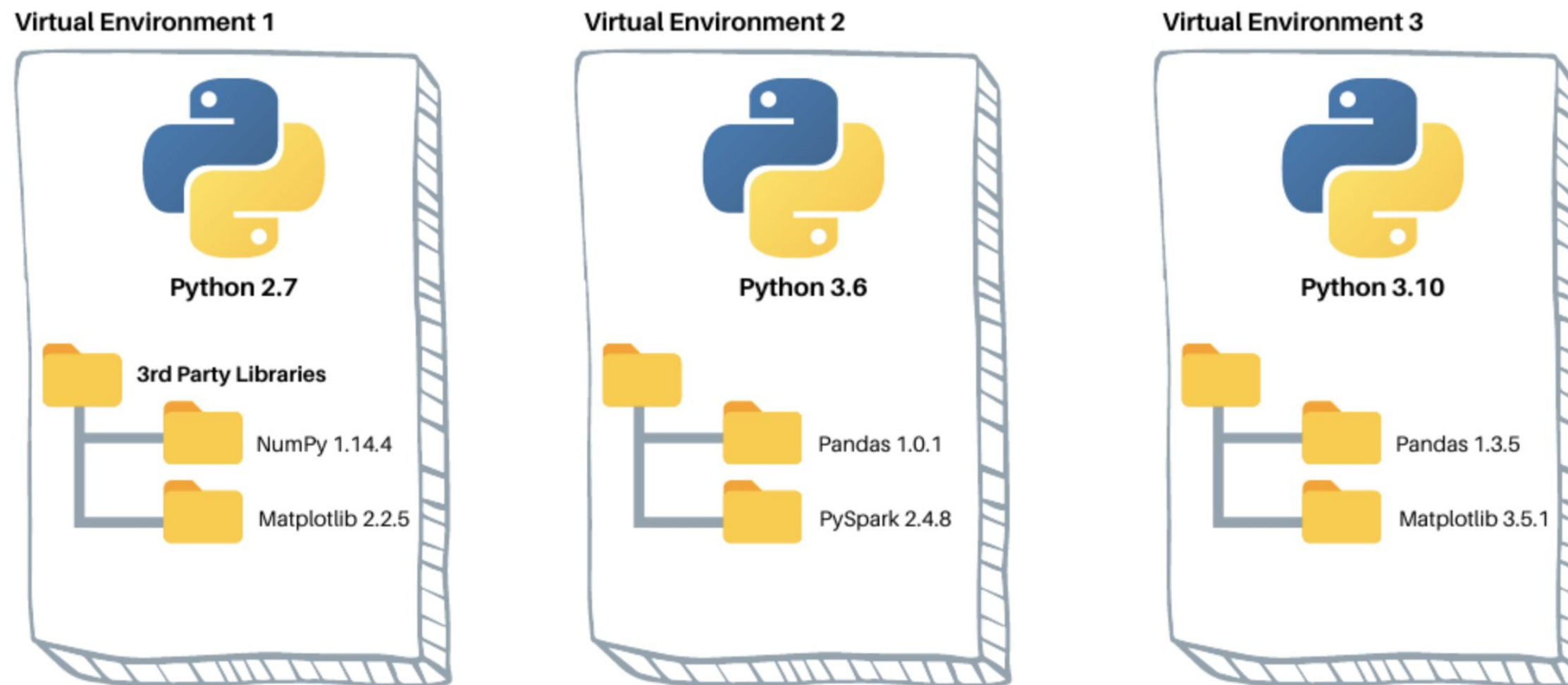
Runtime

- To execute your code you need some runtime
- Python is a dynamic language, so we have many versions
- But we have many projects
- And we need to run it on the same machine

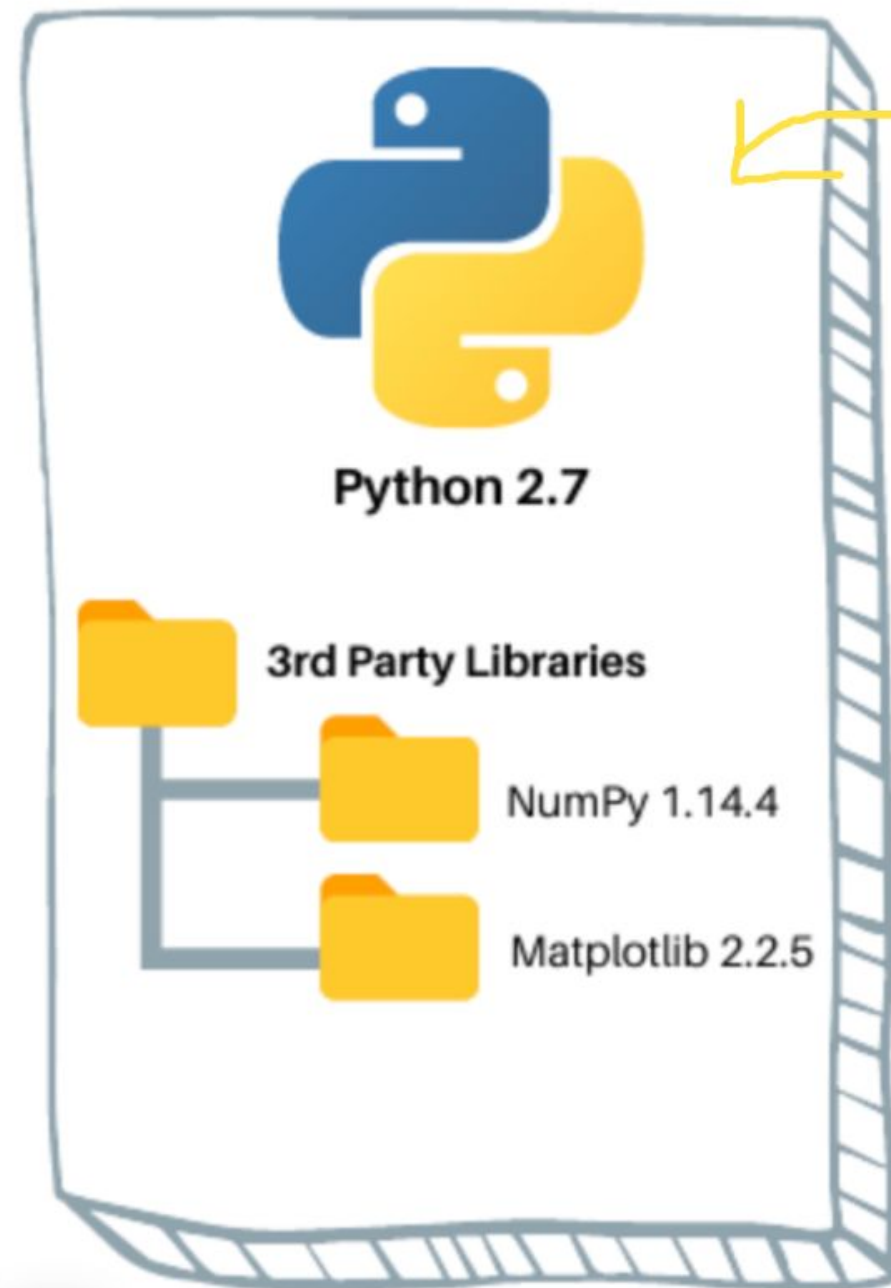


What do we want from environments?

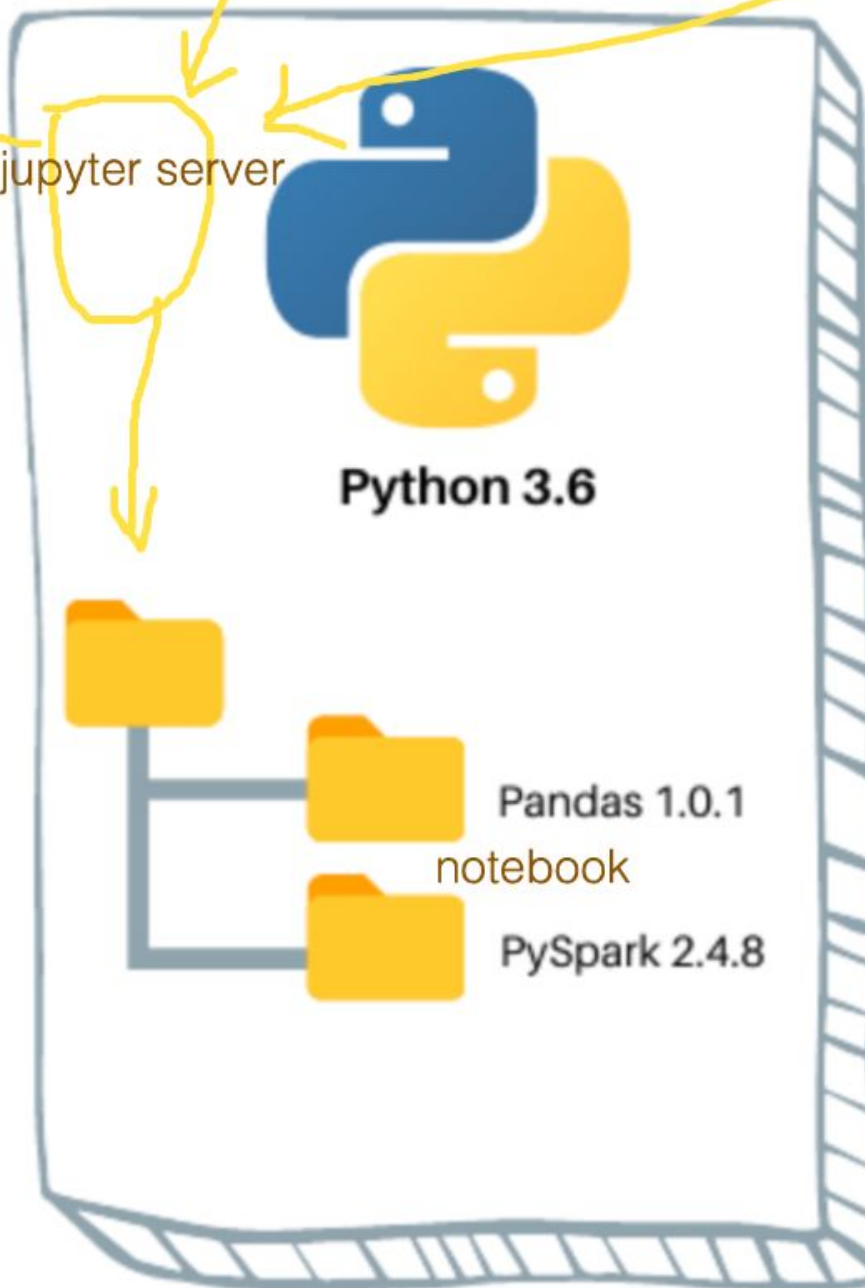
- Create environments
 - separate for each project on one host machine
- Install and remember packages
 - repeatable set of packages for each project



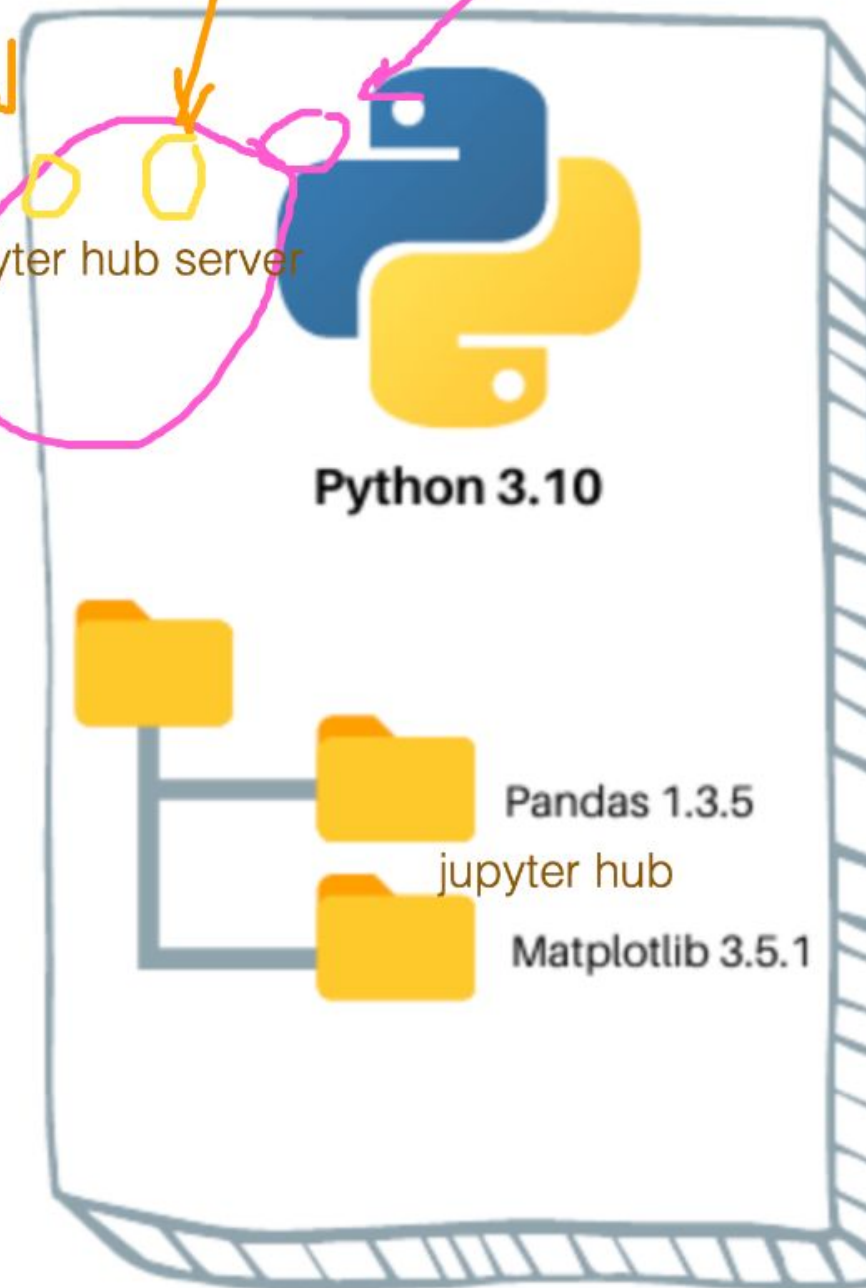
Virtual Environment 1



Virtual Environment 2



Virtual Environment 3



Vanilla solution for environments

- Create environments
 - `python -m venv /path/to/new/virtual/environment`
- Remember and install packages
 - `pip freeze > requirements.txt`
 - `pip install -r requirements.txt`



Vanilla solution for environments

Any problems with this?

- python version not saved
- environments are switched manually
- long and structureless requirements file
 - no hierarchy
- no integrity checks
- no groups of dependencies (prod vs dev)
 - no optionals
- not portable to all OSes
- only python dependencies are managed
- speed of package resolution

Non Python dependencies

1. CUDA
 - a. for most Deep Learning projects
2. Audio codecs for torchaudio
 - a. also for video (ffmpeg)
3. Bag file readers for ROS

Managing environments

- [venv](#) - Python standard library
- [poetry](#)
- conda - able to install non python dependencies, many Python builds
 - [miniconda](#)
 - [mamba](#)
- [uv](#) - super fast tool written in Rust
- [virtualenv](#) - extended toolkit for virtual environments
- [pyenv](#) - automatic envs switching ([good article](#))
- [pipx](#)
- [pipenv](#)

[More on virtual envs](#)

Environment storage approach

I. per project

/proj A/ pyproject.toml
/proj-a

.venv/

⋮

/proj B/ pyproject.toml
/proj-b

.venv

uv

← poetry → conda

II global storage

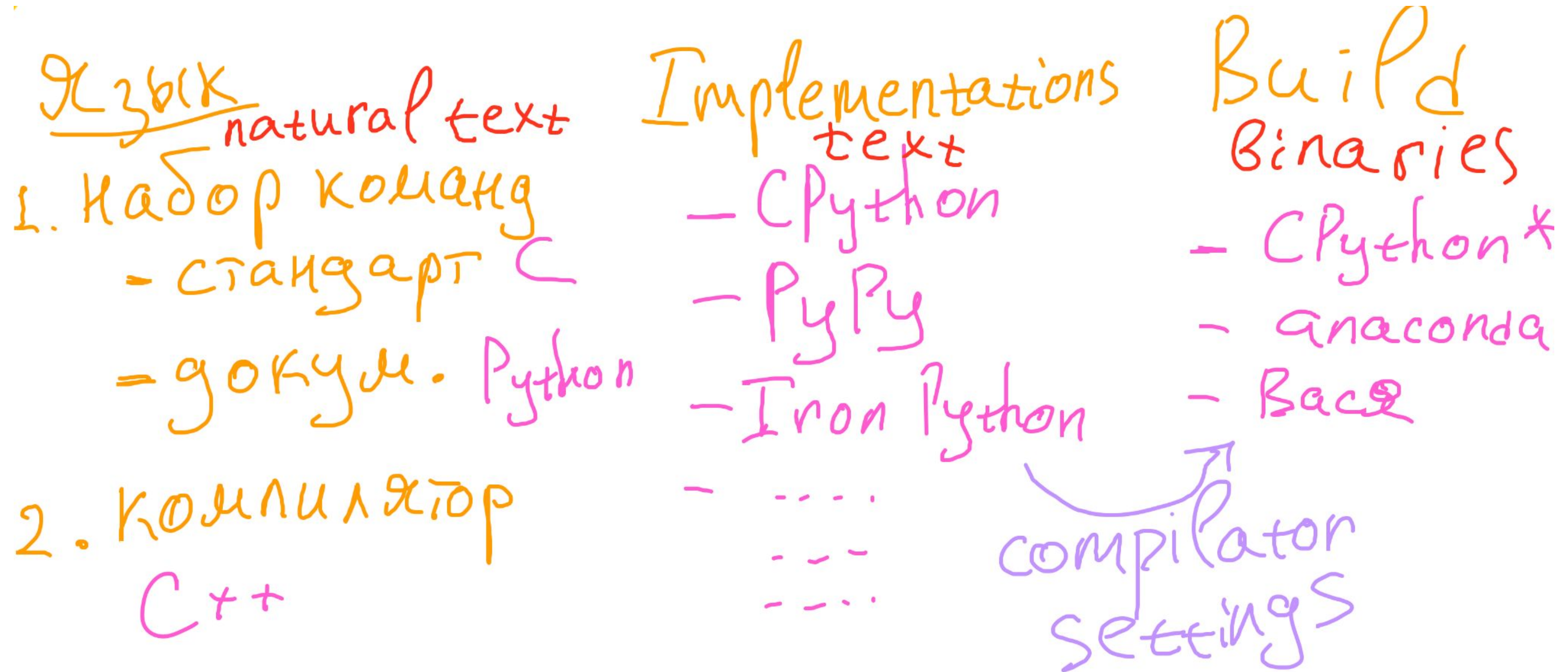
~/conda/envs/env1/

env2/

env3



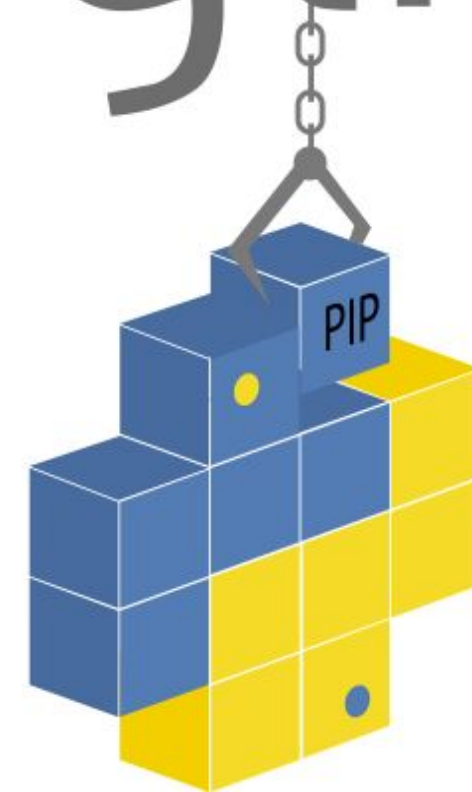
Python implementations



Managing dependencies

- pip (via requirements.txt)
- [poetry](#) - convenient tool covering most needs
- [pip-tools](#)
- [Pipfile](#) (via pipenv)
- conda
 - `conda env export > environment.yml`
- [uv](#) - super fast tool written in Rust

python



Dependency types

- primary
- secondary

uv tree

- `project.dependencies`: Published dependencies.
- `project.optional-dependencies`: Published optional dependencies, or "extras".
- `dependency-groups`: Local dependencies for development.

<https://docs.astral.sh/uv/concepts/projects/dependencies/>

Build systems

- [setuptools](#)
- poetry
- [Hatchling](#) (uv)
- flit
- ...

Poetry capabilities

- dares to install ONE tool per OS
 - also available for CI installation
- manages environments (optional)
- saves Python version
- resolves conflicts better due to hierarchical dependency
- supports groups of dependencies
- manages dependencies for all platforms simultaneously
- additional package indexes made easy

<https://python-poetry.org/docs/>

[Nice tutorial on Poetry packaging](#)



Poetry usage

Creates two files:

- `pyproject.toml` - initial dependencies, [config for all tools](#)
- `poetry.lock` - all concrete dependencies (platform agnostic)

If using conda environments:

```
poetry config virtualenvs.create false
```

For installing dependencies:

```
poetry install numpy
```

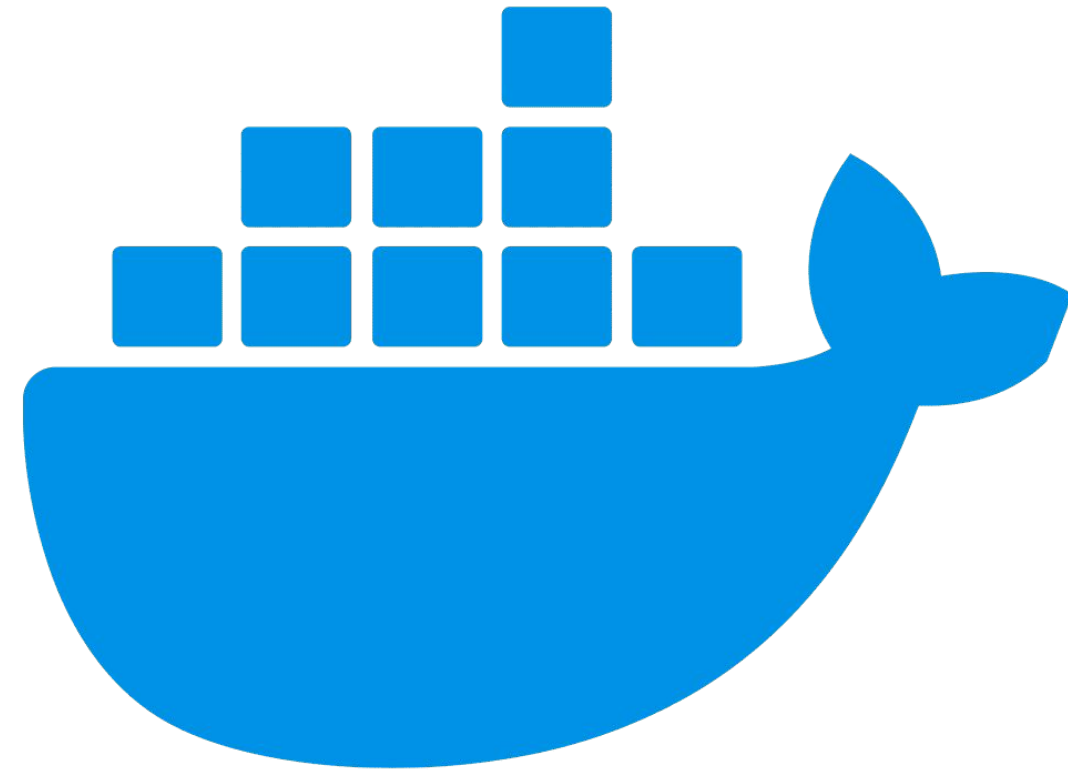
Also it is able to build and publish to PyPI or your own package registry

```
poetry build && poetry publish
```

Docker

As environment for Python:

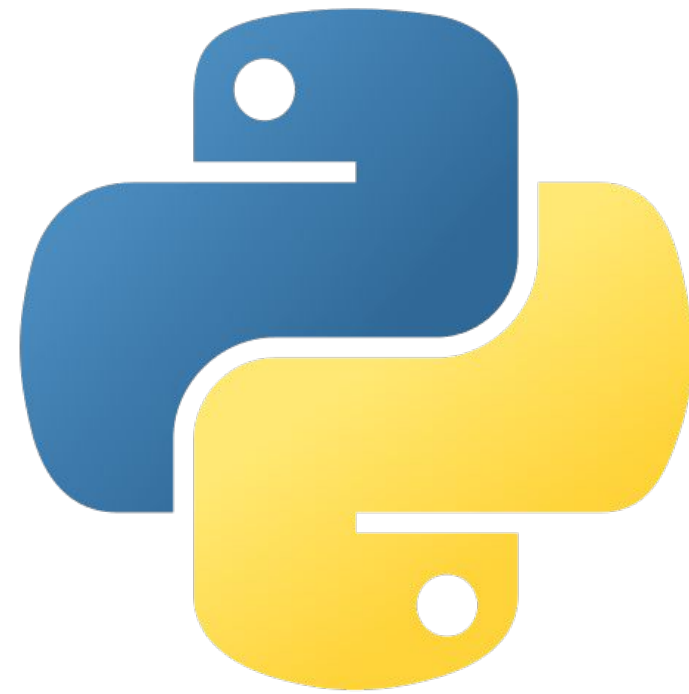
- No change of OS
- No change of Python
- Dependencies compiled into image
- Pure pip can be used



docker®

Package repositories

- PyPI
 - can mirror public version
- custom
 - Artifactory
 - Sonatype Nexus
 - etc.



PyPI

Command Line Interface

girafe
ai

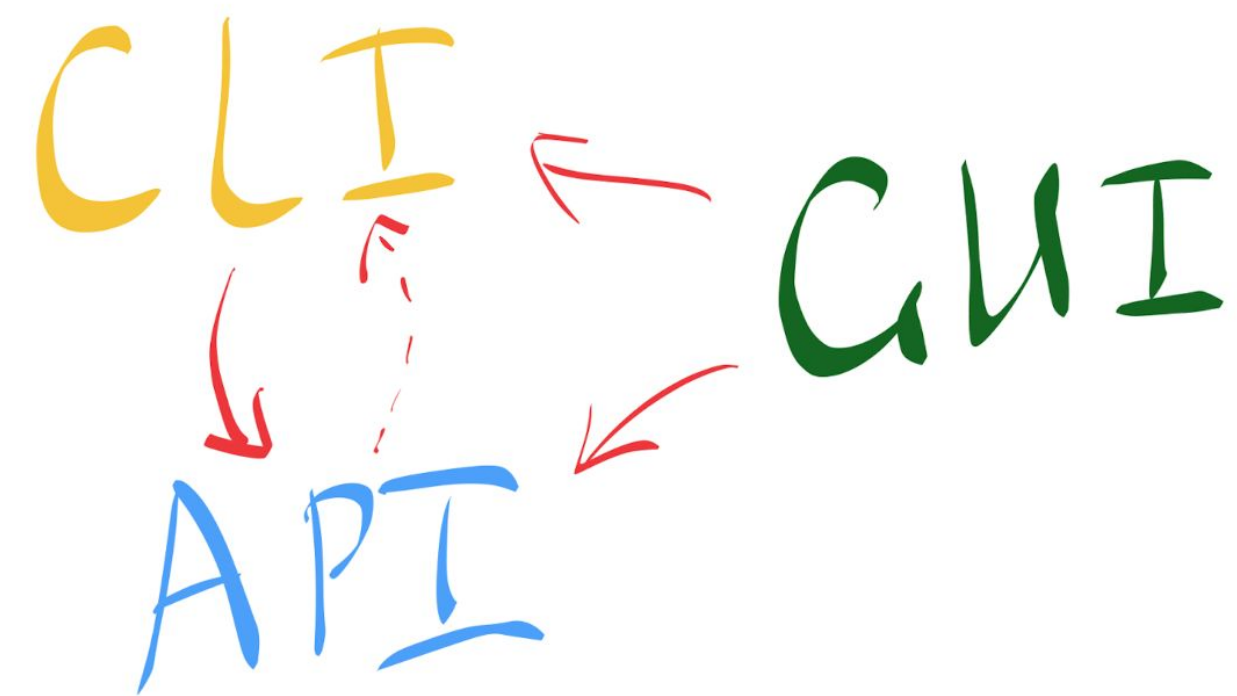
03

CLI definition

Command Line Interfaces (CLI) is a means of interacting with software via commands – each formatted as a line of text.

Command-line interfaces emerged in the mid-1960s, on computer terminals, as an interactive and more user-friendly alternative to the non-interactive mode available with punched cards.

```
[root@localhost ~]# ping -q fa.wikipedia.org
PING text.pmtpa.wikimedia.org (208.80.152.2) 56(84) bytes of data.
^C
--- text.pmtpa.wikimedia.org ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 540.528/540.528/540.528/0.000 ms
[root@localhost ~]# pwd
/root
[root@localhost ~]# cd /var
[root@localhost var]# ls -la
total 72
drwxr-xr-x. 18 root root 4096 Jul 30 22:43 .
drwxr-xr-x. 23 root root 4096 Sep 14 20:42 ..
drwxr-xr-x.  2 root root 4096 May 14 00:15 account
drwxr-xr-x. 11 root root 4096 Jul 31 22:26 cache
drwxr-xr-x.  3 root root 4096 May 18 16:03 db
drwxr-xr-x.  3 root root 4096 May 18 16:03 empty
drwxr-xr-x.  2 root root 4096 May 18 16:03 games
drwxrwx--T.  2 root gdm  4096 Jun  2 18:39 gdm
drwxr-xr-x. 38 root root 4096 May 18 16:03 lib
drwxr-xr-x.  2 root root 4096 May 18 16:03 local
lrwxrwxrwx.  1 root root   11 May 14 00:12 lock -> ../run/lock
drwxr-xr-x. 14 root root 4096 Sep 14 20:42 log
lrwxrwxrwx.  1 root root   10 Jul 30 22:43 mail -> spool/mail
drwxr-xr-x.  2 root root 4096 May 18 16:03 nis
drwxr-xr-x.  2 root root 4096 May 18 16:03 opt
drwxr-xr-x.  2 root root 4096 May 18 16:03 preserve
drwxr-xr-x.  2 root root 4096 Jul  1 22:11 report
lrwxrwxrwx.  1 root root   6 May 14 00:12 run -> ../run
drwxr-xr-x. 14 root root 4096 May 18 16:03 spool
drwxrwxrwt.  4 root root 4096 Sep 12 23:50 tmp
drwxr-xr-x.  2 root root 4096 May 18 16:03 yp
[root@localhost var]# yum search wiki
Loaded plugins: langpacks, presto, refresh-packagekit, remove-with-leaves
rpmfusion-free-updates                                | 2.7 kB    00:00
rpmfusion-free-updates/primary_db                     | 206 kB    00:04
rpmfusion-nonfree-updates                             | 2.7 kB    00:00
updates/metalink                                      | 5.9 kB    00:00
updates                                                | 4.7 kB    00:00
updates/primary_db                                    73% [=====] 62 kB/s | 2.6 MB    00:15 ETA
```



Command Line Interfaces (CLI)

Default tool for CLI in Python is [argparse](#)

Which is clone of C library and is painful to use

```
import argparse

parser = argparse.ArgumentParser(description='Process some integers.')
parser.add_argument('integers', metavar='N', type=int, nargs='+',
                    help='an integer for the accumulator')
parser.add_argument('--sum', dest='accumulate', action='store_const',
                    const=sum, default=max,
                    help='sum the integers (default: find the max)')

args = parser.parse_args()
print(args.accumulate(args.integers))
```

CLIs for humans

Also there are several tools made for humans.

One of them is [Fire](#).

It makes CLI out of pure Python functions and classes.

[Usages](#)

Other options:

<https://click.palletsprojects.com/>

```
import fire

def hello(name="World"):
    return "Hello %s!" % name

if __name__ == '__main__':
    fire.Fire(hello)
```

Then, from the command line, you can run:

```
python hello.py # Hello World!
python hello.py --name=David # Hello David!
python hello.py --help # Shows usage information.
```

CLI packaging

In a modern `pyproject.toml` you may define `[project.scripts]` and `[project.gui-scripts]` section where you define commands to be installed with your package and modules to be involved by that command.

More info [here](#) and [there](#).

```
[project.scripts]
spam-cli = "spam:main_cli"

[project.gui-scripts]
spam-gui = "spam:main_gui"
```

Changelog

Changelog generation:

<https://towncrier.readthedocs.io/en/stable/>

Package versioning

Main method nowadays is **SemVer standard**

<https://semver.org/>

Use for all the projects always!

Exceptions named after date:

- Python Black formatting engine
- YouTube Downloaders (yt-dl, ...)

Reference projects

They are not iconic, just good enough projects!!!

1. Template project with basic setup
<https://github.com/v-goncharenko/data-science-template>
2. Good example of project with most of tools applied
<https://github.com/v-goncharenko/mimics/>
3. Dummy project for this course
<https://github.com/vlad21naumov/cats-and-dogs>
4. Full fledged library made by biologists
<https://github.com/NeuroTechX/moabb>

How to write a code

We will talk about it next time, now some great references:

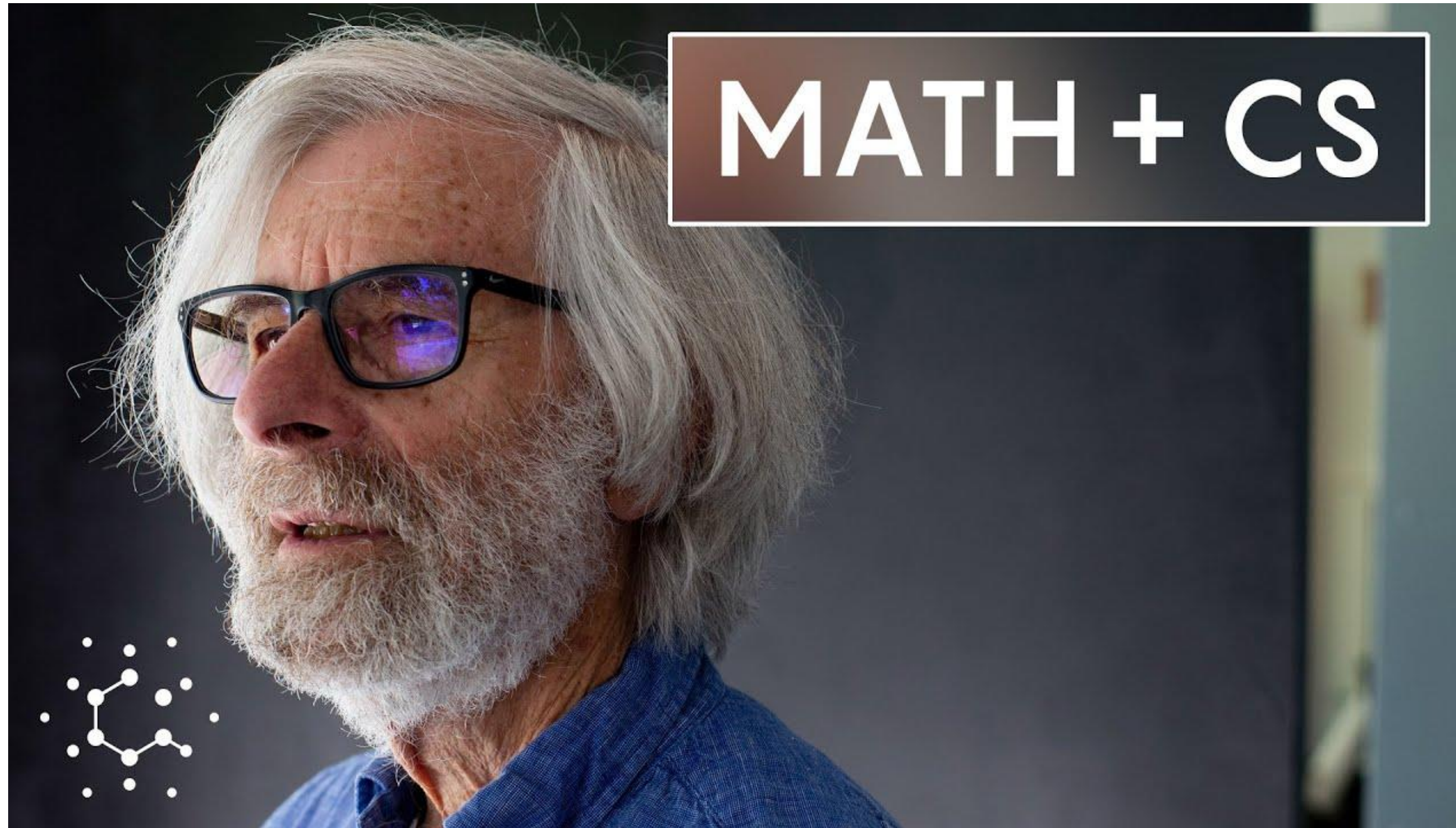
- Summary of the Clean Code book
<https://gist.github.com/wojteklu/73c6914cc446146b8b533c0988cf8d29>
- Same thing adopted for Python
<https://github.com/zedr/clean-code-python>
- And for ML
<https://github.com/davified/clean-code-ml>

There's no silver bullet in code writing,
so despite all the books you need to think yourself.

Programming is not coding

It is an applied math, so underlying algorithms play crucial role in the system

<https://www.youtube.com/watch?v=rkZzg7Vowao>



How to write a code in AI era

- <https://lucumr.pocoo.org/2025/9/29/90-percent/>
 - <https://t.me/seeallochnaya/2952>

TLDR:

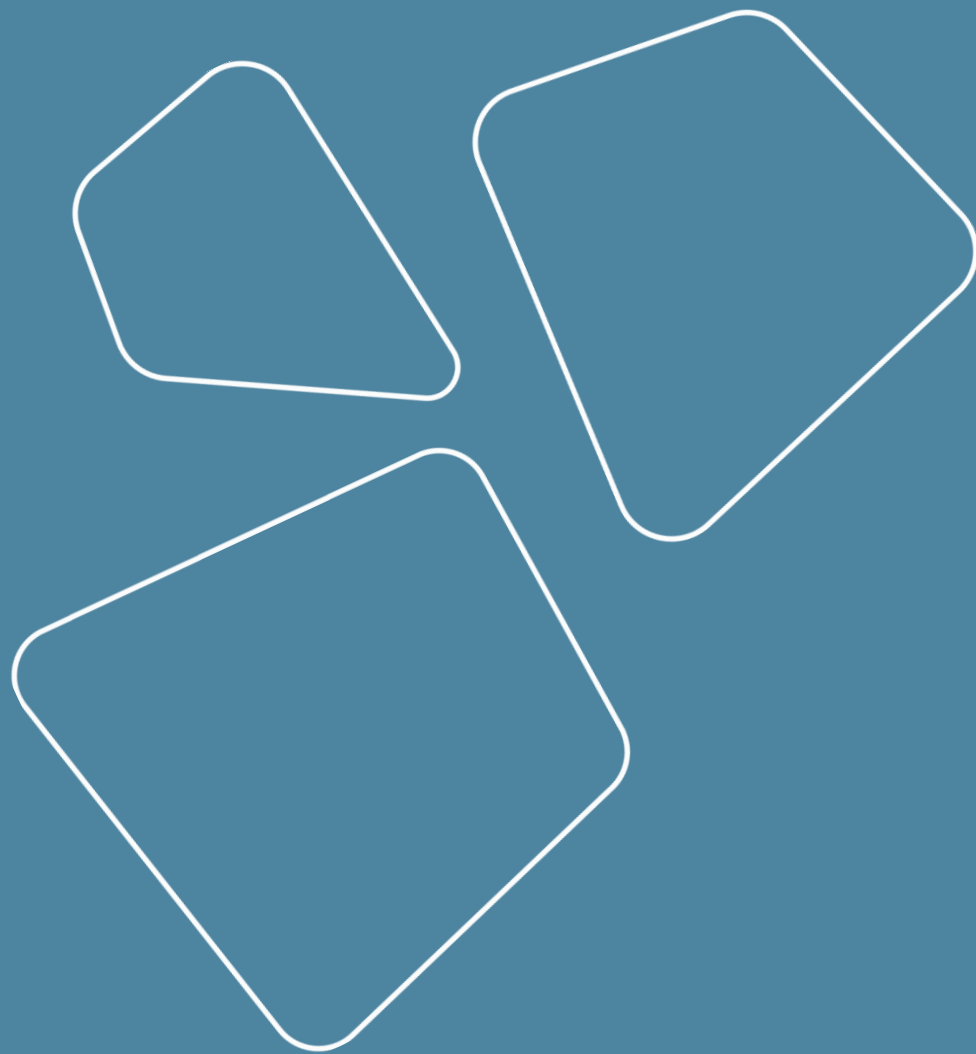
Agents make coding *different*. You are still responsible for result but now you may concentrate on harder parts of your codebase and verify more than boilerplate.

Instruments for AI coding

- **Agents**
 - Codex
 - Claude Code
 - Hermes, OpenCode, Goose (FOSS)
- IDEs
 - Cursor <https://cursor.com/>
 - Windsurf <https://windsurf.com/>
- IDEs extensions
 - Codex ext for VSCode
 - Claude ext for VSCode
- CLIs
 - <https://github.com/Aider-AI/aider>
 - Claude code
- <https://github.com/coderamp-labs/gitingest>
 - LLM friendly preprocessor of code

Revise

1. Python project structure
2. Managing environments
3. Managing dependencies
4. CLI
5. Changelog



Thanks for attention!

Questions?

